



Lista de Exercícios –

Operações *bitwise*

1) Verifique e compreenda o que está sendo feito nas seguintes linhas de código.

- a) `x |= 0x00000004;`
- b) `y &= ~(0x0F000004);`
- c) `port_data_out &= ~(1 << 17);`
- d) `PDOR &= 1 << 5;`
- e) `g_task[i] = 1 << 5;`
- f) `GPIOB.data = ~(1 << 12);`
- g) `GPIOD->PTOR |= (1 << 5) | (1 << 2);`
- h) `z &= ~(1 << 7 | 1 << 4);`
- i) `omega |= 0x20000000 | 0x00080000 | 0x00000004;`
- j) `c &= ~(1 << 7) & ~(1 << 4) & ~(1 << 1);`

2) Desenvolva uma função que retorne o número de bits (0 ou 1) que um número possui, com o seguinte protótipo:

```
int bit_count( int num, enum bit_count_opt opt);
```

Onde:

- num – é o número que será avaliado;
- bit_count_opt – BIT_COUNT_OPT_ZERO, se será contado o número de zeros;
BIT_COUNT_OPT_ONE, se será contado o número de uns.

3) Implemente um programa que troca os *nibbles* de um byte. Por exemplo, 0xAB deve se tornar 0xBA.

4) Implemente um programa que troca os *bytes* de uma palavra de 32 bits. Por exemplo, 0xABCD deve se tornar 0xCDAB.

- 5)** Utilizando operadores bitwise, crie uma função capaz de imprimir um número em formato binário.

Exemplo:

```
Imprime(15); // deve imprimir 1111
```

- 6)** Crie uma função que receba uma string com a representação de um número em binário e retorne este número em decimal.

Exemplo:

```
int i = Decimal("1010"); // deve atribuir 10 para a variável i
```

- 7)** Escreva um programa que fique indefinidamente pedindo ao usuário, um número inteiro de 32 com sinal e uma das seguintes operações *bitwise* para serem feitas com ele:

- a) NOT
- b) Deslocamento de n bits para a esquerda
- c) Deslocamento de n bits para a direita
- d) Aplicação de máscara de bits

Deve ser apresentando na saída padrão o valor binário do número inserido em 32 bits, além do resultado da operação em decimal e em binário de 32 bits.

Alguns exemplos:

Digite o valor: 10 Binário: 0000 0000 0000 0000 0000 0000 0000 1010 Indique a operação a ser realizada: a) NOT b) Deslocamento de n bits para a esquerda c) Deslocamento de n bits para a direita d) Aplicação de máscara de bits b Indique o número de bits de deslocamento: 3 Binário: 0000 0000 0000 0000 0000 0000 0101 0000 Decimal: 80	Digite o valor: 10 Binário: 0000 0000 0000 0000 0000 0000 0000 1010 Indique a operação a ser realizada: a) NOT b) Deslocamento de n bits para a esquerda c) Deslocamento de n bits para a direita d) Aplicação de máscara de bits d Indique máscara: 80 Binário: 0000 0000 0000 0000 0000 0000 0101 1010 Decimal: 80
--	--

- 8) Dada uma struct genérica, crie um programa que imprima na saída padrão todos os bytes de uma variável tipada com essa struct, na ordem do primeiro elemento ao último elemento. Cada byte será apresentado em uma linha separada.

Exemplo:

Dada a struct,

```
struct S
{
    int x : 3;
    unsigned y : 2;
    char c : 5 ;
    short s;
};
```

Se para uma variável tipada com essa struct, $x = -1$, $y = 2$, $c = 30$ e $s = 8$, a saída deverá ser:

```
11101010
11111111
10000110
00000000
01111100
11011000
00010000
00000000
```

Dica: utilize uma Union para auxiliar na impressão dos bits!

- 9) Escreva um programa que calcule a distância de Hamming entre dois bytes e armazene o resultado como um número inteiro positivo (8 bits) na memória. A distância de Hamming entre dois bytes quaisquer é definida como sendo o número de bits que devem ser alterados em um dos bytes para transformá-lo no segundo byte, ou seja, o número de bits diferentes entre estes dois bytes. A operação booleana "Ou-Exclusivo" entre dois bits é verdadeira quando estes bits são diferentes, e é falsa quando os bits são iguais.